

Projet GNC Quadrotor Outdoor — Document maître v3.1

Version 3.1 du 12 septembre 2026 (v3 = refonte, v3.1 = cinq ajustements après relecture externe, §0) — périmètre recentré sur la navigation INS/GNSS embarquée et une loi de commande en vol ; MATLAB, MPC et guidage sortis du plan. Remplace la v2.2 (archivée dans [docs/archive/](#), où restent les énoncés complets des TP retirés). Les « §n » renvoient à la Partie I, les [n] à la bibliographie du §12.

Sommaire. Partie I : 0. Historique · 1. Fiche d'identité · 2. Compétences · 3. Architecture · 4. Plan · 5. Questions d'entretien · 6. Environnement · 7. Achats · 8. Protocole terrain · 9. Métriques · 10. Risques · 11. Extensions · 12. Références. Partie II : mode d'emploi · vue des sprints · S0 · S1 · S2 · S3 · S4 · S5 · S6 · S7 · S8 · S9 · cours · annexes.

Décisions figées : terrain 2 ha en A3 · Holybro X500 V2 ARF · Pixhawk 6C Mini + M10 + H-Flow (reçus le 12 sept) · PX4 v1.17.0 figé · ESKF INS/GNSS loosely coupled avec repli GNSS-denied, **en source sur la carte** · module de commande custom à l'étage **position/vitesse**, boucles internes stock · PID en vol, LQR/LQI en simulation (vol opportuniste), ESO seul observateur · tout en Python + C++, **pas de MATLAB** · 30 h/semaine + candidatures, 9 sprints, fin visée **24 janvier 2027**, tampon jusqu'à fin février · Coursera Toronto en S3-S4 · pas de télémétrie, logs SD · balise de signallement du commerce.

Partie I — Cadrage

0. Historique des décisions

- **v1 → v2 (2 sept)** : indoor → outdoor sur terrain A3 ; micro-quad custom → X500 V2 (zéro soudure, plateforme de référence PX4) ; flow GPS-denied → INS/GNSS loosely coupled avec repli flow/ToF ; mocap → hold-out GNSS ; sprints de 2 semaines avec Go/No-Go.
- **v2.2 (12 sept)** : semaine du 9 au 12 non travaillée → calendrier décalé d'une semaine ; carte reçue le 12 → TP 0.4 (flash, vérification M10/H-Flow, logs Allan) en S0.
- **v3 (12 sept) — refonte.** Constat : le périmètre v2 (ESKF complet + trois lois de commande + MPC embarqué + FSM + A*/RRT* + campagne factorielle) n'était pas tenable en 5 mois à une personne, et il manquait le TP qui met l'ESKF en source sur la carte. Décisions : (1) **cible principale navigation** ; (2) MATLAB supprimé — le PFE Simulink couvre déjà le mot-clé, le workflow MBD se pratique en Python → C++ ; (3) MPC et guidage sortent du plan (§11), à définir en fin de projet ; (4) le module de commande custom passe à l'**étage position/vitesse** : plus sûr pour le module de commande (une erreur fait dériver, pas retourner — l'estimateur en source, lui, expose aussi les boucles internes par l'attitude qu'il publie, d'où l'ordre A-logiciel → vols en parallèle → B-vol), consomme l'état de ton ESKF, même étage pour PID et LQR, perturbations de force (vent, masse) là où l'ESO agit ; (5) observateur : ESO seul, comparé à l'action intégrale ; (6) nouveau TP 3.9 « ESKF sur cible » ; (7) jalons scindés logiciel/plateforme, campagne cumulative, README à chaque jalon, CI minimale ; (8) corrections théoriques issues d'une revue externe (observabilité du cap, biais non identifiables en statique, retards, NIS avant gating). Résultat : 9 sprints, fin le 24 janvier, cinq semaines de tampon pour l'hiver.
- **v3.1 (12 sept)** — cinq ajustements après relecture externe : (1) le portage C++ du filtre complet est affecté explicitement aux TP 3.1-3.2 (+10 h), et le TP 3.9 est découpé en 3.9a interface + parallèle et 3.9b source + boucle fermée (18 h) ; le TP 3.8 passe en S4 pour équilibrer ; (2) l'estimation du vent à bord (fusion de traînée EKF2) n'existe pas sur les vols où ton ESKF est en source — le 4.6b vole sur EKF2, la mission finale s'appuie sur la station et des vols stock alternés ; (3) les jalons C et D n'imposent plus la conclusion de

l'expérience ESO : « comparaison reproductible et choix justifié », puis « mission avec le contrôleur retenu » ; (4) NIS : dépassements des bornes basse et haute examinés séparément, bornes χ^2 à la dimension de la mesure, référence [8b] ; (5) perte de position : COM_POSCTL_NAVL choisi explicitement et vérifié en SITL — avec 1, une hauteur invalide coupe les moteurs ; (6) la configuration retenue au jalon C (PID ou PID + ESO) est **qualifiée sur ton ESKF**, en SITL puis en vol court, au début du 7.1 : le changement d'estimateur change le bruit et le retard de la vitesse que voient le contrôleur et l'ESO.

1. Fiche d'identité

Navigation inertielle INS/GNSS embarquée et commande de position d'un quadrotor en extérieur : ESKF multi-capteurs avec repli GNSS-denied, tournant en source sur STM32H7, validé par rejeu et hold-out ; boucle position/vitesse PID/LQR avec rejet de perturbation par ESO, comparée au stock en campagne d'essais.

Durée / rythme	9 sprints de 2 semaines (2 sept → 24 jan), 30 h/semaine projet + 5-8 h candidatures, tampon jusqu'au 28 février
Plateforme	Holybro X500 V2 ARF (~1,4 kg, 12-18 min par accu), Pixhawk 6C Mini (STM32H743, NuttX), PX4 v1.17.0 figé
Capteurs	2 IMU (ICM-42688-P, BMI055), baro, mag IST8310, GNSS u-blox M10 (5-10 Hz), Holybro H-Flow DroneCAN (flow PAA3905 + ToF 0,08-30 m)
Cœur	ESKF 16 composantes d'erreur (17 nominales) INS/GNSS loosely coupled + baro/mag, repli flow/ToF ; en parallèle puis en source sur le H7 ; rejeu vs EKF2, hold-out, NIS/NEES
Contrôle	Module PX4 position/vitesse : cascade PID industrielle en vol ; LQR/LQI en simulation ; ESO vs action intégrale face au vent et à la masse ajoutée — configuration retenue sur preuves
Outils	C++ (core/ portable + module PX4), Python (prototypes, rejeu, synthèse), SITL Gazebo, ulog/Flight Review, GoogleTest, CI minimale
Zone d'essais	Terrain dégagé de 2 ha, A3, VLOS, 2-15 m d'altitude en pratique ; jardin réservé au banc
Budget	~330 € engagés (carte, M10, H-Flow) + ~710-770 € commande 2 ≈ 1 050-1 100 € ; cellule HX711 ~7 €
Contrainte	Aucune invention : méthodes et workflows réellement utilisés en industrie, transférables en entretien

Pitch CV visé (à réécrire avec tes chiffres, en distinguant simulation / rejeu / cible / vol) :

Projet personnel (5 mois) — navigation embarquée d'un quadrotor outdoor sous PX4 (X500, Pixhawk 6C Mini) : conception, implémentation C++ (STM32H7/NuttX) et validation en vol d'un ESKF INS/GNSS loosely coupled (IMU, GNSS, baro, mag) avec repli GNSS-denied par flow optique et ToF, utilisé comme source de navigation, évalué par rejeu contre EKF2, hold-out GNSS et analyse de consistance (NIS/NEES). Boucle de commande position/vitesse maison (PID, LQR), rejet de perturbation par observateur d'état étendu évalué contre l'action intégrale, comparée au contrôleur stock en campagne d'essais avec métriques automatisées.

2. Compétences visées, lecture franche

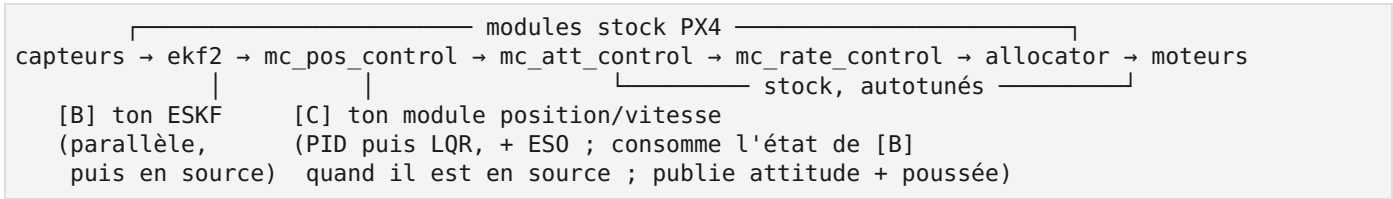
Volet	Ce que le projet t'apprend	Difficulté	Poids en entretien GNC
Navigation	Conventions et quaternions · variance d'Allan · ESKF · couplage lâche INS/GNSS (retard, bras de levier, R adaptatif) · modèles baro/mag/flow/ToF · perte GNSS et bascule de mode · NIS/NEES, gating · rejeu, hold-out · exécution sur cible	Élevée (ESKF, modèles de mesure)	Le cœur : « expliquez un EKF », « loosely vs tightly », « que faites-vous quand le GNSS décroche », « comment réglez-vous Q et R », latence des mesures
Contrôle	Cascade PID industrielle (anti-windup, feedforward) · linéarisation, discrétisation, LQR/LQI · ESO vs intégrateur	Moyenne	« PID vs LQR », saturations, rejet de perturbations non modélisées
Embarqué	NuttX uORB cross-compilation code sans	Moyenne	« Avez-vous fait du temps réel »

	fast, done, cross compilation, code sans allocation, profilage on-target, DroneCAN	rejeu à élevée	« Avec vous fait du temps réel ? », du modèle au code
Process	Prototype Python → C++ back-to-back · tests · rejeu · campagnes terrain (plan de vol, météo, répétitions, rapport) · README et reproduction à chaque jalon	Faible à moyenne	Cycle en V, culture de validation, sens du terrain

Lucidité : le quotidien du GNC drone, c'est un EKF fourni, un PID, beaucoup d'intégration et de validation. Le projet couvre ce quotidien (rejeu, tests, terrain) et la couche au-dessus (ESKF en source). Les items « faciles » — rejeu, tests, campagne — sont ceux qui pèsent le plus pour un premier poste ; ne les sacrifie jamais à un algorithme de plus.

3. Architecture technique

3.1 Chaîne PX4 et points d'insertion



Chaque brique custom est un **module C++ PX4** abonné aux mêmes topics que le module stock et publiant les mêmes sorties. Le stock est désactivé au démarrage (script SD) et rebasculé par **interrupteur radio** — mécanisme conçu et testé au TP 4.2, pas une intention. La comparaison « ma brique vs le stock » sur les mêmes vols est le fil rouge.

3.2 Dépôt

```

quad-gnc/
├── core/           # C++ portable : ESKF, PID, LQR, ESO – zéro dépendance PX4, zéro allocation
│   ├── include/core/
│   ├── src/
│   └── tests/
├── px4/           # modules PX4 (wrappers uORB), config carte, scripts de démarrage, jeux de paramètres
├── replay/        # rejeu uLog : pyulog + binding vers core/, hold-out
├── sim/           # simulateur Python, capteurs synthétiques, prototypes, synthèse PID/LQR
├── flight-tests/  # plans de vol, logs, params par vol, météo, rapports
├── docs/          # journal, conventions, dérivations, rapports, terrain, mass-budget, archive/
└── .github/       # CI minimale : cmake + ctest sur chaque push
  
```

Principe : le même core/ tourne partout — tests PC, rejeu, SITL, H7. Une implémentation, quatre niveaux de validation.

3.3 Niveaux de validation, du moins cher au plus cher

1. **Tests unitaires** (GoogleTest, `ctest` vert avant chaque commit, CI sur push).
2. **Back-to-back** : le prototype Python est l'oracle ; le C++ rejoue les mêmes vecteurs à tolérance près. Ça prouve la fidélité du code au modèle, pas la justesse du modèle.
3. **Rejeu de logs réels** : ton ESKF contre EKF2 image par image ; hold-out GNSS sur fenêtres. Cohérence avec une référence embarquée, pas erreur vraie.
4. **Simulation avec vérité** : simulateur Python à biais connus (NEES possible sur tous les états) ; SITL Gazebo (boucle fermée, perte GNSS injectable, vérité position/attitude).
5. **Banc → vol** : quand tout le reste est vert, et quand la météo le permet.

3.4 Navigation outdoor

Le modèle de propagation est la cinématique de l'IMU, pas la dynamique du drone. L'ESKF intègre gyro et accéléro et fonctionnerait tel quel dans une voiture ; masse, inertie et poussée servent au contrôle et au simulateur.

Un filtre, deux modes. État nominal : position, vitesse, quaternion, biais gyro, biais accéléro, biais baro (17 composantes) ; erreur : 16. Ce qui change entre modes, c'est

l'ensemble des mesures actives.

Mode	Mesures fusionnées	Observable	Dérive
GNSS (nominal)	GNSS position + vitesse NED, baro, mag ; flow/ToF surveillés, non fusionnés	Position, vitesse, attitude ; biais accéléro et cap sous excitation (déplacements, virages) — en hover prolongé, le cap ne tient que par le mag et les biais convergent lentement	Rien à long terme ; précision bornée par le GNSS (~1-2 m, ~0,1 m/s)
GNSS-denied (repli)	Flow + ToF + baro + mag	Vitesse, hauteur sol, attitude, cap	Position horizontale : marche aléatoire, c'est la métrique du hold-out
IMU seule	Aucune	Rien	Tout — mesuré au TP 3.1

Points de modélisation (chacun un item du TP 3.2) :

- **GNSS position** : NED local, origine figée à l'armement ; bras de levier antenne-IMU ($p_{gnss} = p + R \cdot l$) ; R adaptatif depuis hAcc/vAcc ; **retard 100-200 ms** compensé par tampon d'états et de covariance (fusion à l'instant de validité, correction propagée au présent — le mécanisme d'horizon retardé d'EKF2).
- **GNSS vitesse** : velNED du u-blox, meilleure que la dérivée de position ; c'est elle qui fixe les biais accéléro.
- **Cap** : alignement initial par le mag (déclinaison) ; en déplacement, le lacet est aidé par la cohérence entre l'accélération que prédit l'attitude et la vitesse GNSS mesurée — **pas par la route sol**, qui n'est pas le cap d'un quadrotor capable de translation latérale.
- **Baro** : biais dans l'état (météo, souffle des hélices). **Flow** : taux de défilement, rotation propre retirée, mis à l'échelle par la hauteur sol estimée. **ToF** : distance le long de l'axe, compensée de l'inclinaison, sol plat.
- **Bascule** : sur perte de fix ou gating GNSS répété, sans reset de P ; retour avec réinitialisation contrôlée de la position (saut documenté). La logique de mode, ses resets et sa validité d'état : c'est ta machine à états.

Vérité terrain. Sans RTK, la référence est EKF2 avec GNSS ; ton ESKF et EKF2 partagent les capteurs, donc certaines erreurs. Dis-le en entretien : en mode GNSS tu mesures une cohérence ; en simulation à biais connus tu mesures une erreur.

4. Plan par sprints

4.1 Rythme

- **Capacité** 60 h par sprint ; charge planifiée ~40 h — la marge est délibérée : elle absorbe l'intégration, la météo et les candidatures (5-8 h/semaine, créneau fixe).
- **Capacité variable** : toutes les semaines ne font pas 30 h. Une semaine courte se rattrape dans le sprint ; c'est le **bilan de sprint** qui décale ou ravance le calendrier, jamais une semaine isolée.
- **Semaine type** : 4 jours projet (matin papier et lecture, après-midi implémentation, tests, rejeu), 1 jour candidatures + journal + bilan.
- **Les vols se déclenchent sur fenêtre météo**, jamais sur le calendrier : vent < 15 km/h pour le réglage, 15-30 km/h pour le rejet de perturbation, jamais sous la pluie. Chaque jour volable, tu voles **tous** les scénarios du plan de vol, y compris ceux des TP à venir. Les semaines de pluie sont des semaines de rejeu.
- **Règles** : jalon vert avant nouvelle feature ; un TP qui dépasse $\times 2$ son estimation est un signal d'arbitrage, pas une honte ; **si l'incertitude principale d'un TP ne se résorbe plus, on arbitre tout de suite**, sans attendre le $\times 2$.
- **Chaque jalon publie** : tag git, README court, une figure expliquée, la commande qui la reproduit, une difficulté réellement résolue. C'est ce que tu montres en entretien, pas le

jalón final.

4.2 Les sprints

Sprint	Dates	TP	Charge bureau	Sortie / jalón	État au 12 sept
S0	2 → 20 sept	0.1 · 0.2 · 1.1 · 1.2 · 0.4 (carte : flash, M10/H-Flow, logs Allan) · 1.3	~51 h	carte vérifiée, commande 2 passée, simulateur Python testé	0.1, 0.2, 1.1 faits ; 1.2 en cours (étape 2) ; 0.4, 1.3 prévus
S1	21 sept → 4 oct	2.1 · 1.5 · 2.2 · 2.5a (squelette de rejeu) · 3.4 (à réception) · 0.3	~48 h	rapport Allan, Kalman 1D et Mahony, drone en phase 0, balise déclarée	prévu
S2	5 → 18 oct	2.3 · 2.4 · 3.7 · 3.5 · 2.5b	~57 h	A-logiciel (attitude en rejeu) · A-plateforme (vols stock)	prévu
S3	19 oct → 1er nov	3.0 · 3.1 (dérivation + C++) · 3.2a (modèles + C++) · Toronto mod. 1-3	~40 h + 14 h	GNSS/baro/mag validés en simulation	prévu
S4	2 → 15 nov	3.2b · 3.3 · 3.6 · 3.8 · Toronto mod. 4-5 + projet	~38 h + 13 h	B-rejeu (navigation complète en rejeu et simulation)	prévu
S5	16 → 29 nov	3.9a-b (ESKF sur cible) · 4.1 · 4.3	~34 h + vols	ESKF en parallèle sur le H7, PID synthétisé, banc fait	prévu
S6	30 nov → 13 déc	4.2 · 4.4	~28 h + vols	B-vol (ESKF en source) · C-PID (ton module vole)	prévu
S7	14 → 27 déc (Noël)	4.5 · 4.6a (ESO en simulation)	~20 h	LQR/LQI et ESO validés en simulation	prévu
S8	28 déc → 10 jan	4.6b (vols) · LQR en vol si possible	~14 h + vols	C (rejet de perturbation chiffré)	prévu
S9	11 → 24 jan	7.1 (qualification sur ESKF, campagne cumulative, rapport, README, CV)	~26 h + vols	D — fin de projet	prévu
Tampon	25 jan → 28 fév	rattrapage météo, finitions, entretiens ; extensions (§11) seulement si D est vert	—		

S2 est le sprint dense ; soupape : 2.5b glisse en S3 et A-logiciel se prend sur logs bureau + premier vol.

4.3 Jalons Go/No-Go

A-logiciel (18 oct) — `ctest` vert, CI verte ; en rejeu sur logs bureau et premiers vols, attitude ESKF vs EKF2 : RMSE roulis/tangage < 1° en statique, < 3° en vol calme ; biais gyro convergés, cohérents avec Allan ; désaccord de cap expliqué (non observable sans mag). **A-plateforme (18 oct)** — X500 vole en stock en Position GNSS ; autotune stock exécuté = référence ; failsafes testés (jeu « vol libre ») ; ≥ 3 vols logués avec paramètres exportés et météo.

B-rejeu (15 nov) — Simulation Python à biais connus : NEES cohérent sur les états dont la vérité est connue. SITL : NIS **calculé avant gating**, bornes χ^2 à la dimension de chaque mesure ; **les deux côtés examinés séparément** : part au-dessus de la borne haute (incertitude sous-estimée ou erreur de modèle) et part au-dessous de la borne basse (incertitude surestimée), chacune de l'ordre de quelques % (échantillons corrélés : lis la tendance, pas le compte exact) ; couverture indicative ~90-98 % ; taux de rejet reporté, RMSE vitesse < 0,15 m/s, perte GNSS injectée gérée. Réel : mode GNSS cohérent avec EKF2 sur un vol complet, innovations blanches ; hold-out sur **fenêtres non chevauchantes** de

vols réservés à l'évaluation : dérive médiane < 3 m à 60 s (indicatif, à recalibrer).

B-vol (13 déc) — ESKF en source sur le H7 : le drone tient un hover et un carré en mode Position sur ton estimateur ; temps d'exécution mesuré (moyen, max observé, échéances manquées) ; bascule EKF2 $\rightleftharpoons$ ESKF au sol par script de démarrage ; action de `COM_POSCTL_NAVL` choisie et vérifiée en SITL sur tes drapeaux de validité (§8.3) ; cohérence d'attitude bord ESKF/EKF2 démontrée en parallèle avant le passage en source. **C-PID (13 déc)** — Ton module position/vitesse vole (hover, échelons, carré) sur EKF2 puis sur ton ESKF, performances comparables au stock (± 20 %) ; back-to-back Python/C++ vert ; bascule stock $\rightleftharpoons$ custom testée sous consigne non nulle.

C (10 jan) — Comparaison **reproductible** PID vs PID + ESO sous masse ajoutée 200 g et vent logué : gains et dégradations quantifiés (écart max, temps de récupération, sensibilité au bruit, $\int u^2$), **choix de configuration justifié** — garder ou écarter l'ESO sont deux résultats valides ; LQI vs LQR + ESO en simulation ; LQR en vol si une session l'a permis.

D (24 jan) — Mission QGC standard (décollage $\rightarrow$ 3-4 waypoints $\rightarrow$ atterrissage) volée 3 fois sur ton ESKF en source et le **contrôleur retenu au jalon C** (PID ou PID + ESO), **qualifié sur ton ESKF en SITL puis en vol court avant la campagne**, comparée au stock sur les mêmes vols ; campagne cumulative avec métriques par script ; rapport, README, vidéo, CV mis à jour.

5. Questions d'entretien

1. « EKF vs ESKF — pourquoi l'error-state ? » $\rightarrow$ S2.
2. « Comment réglez-vous Q et R ? » $\rightarrow$ Allan, précisions u-blox, NIS (S1-S4).
3. « Loosely vs tightly coupled : lequel, pourquoi ? » $\rightarrow$ S3 (le M10 ne sort pas de pseudo-distances).
4. « Le GNSS décroche : combien de temps tenez-vous ? » $\rightarrow$ hold-out, bascule, dérive chiffrée (S4).
5. « Comment gérez-vous la latence des mesures ? » $\rightarrow$ tampon d'états et de covariance (S3).
6. « Comment validez-vous sans vérité terrain ? » $\rightarrow$ rejeu, cohérence, hold-out, NEES en simulation (S4).
7. « Votre estimateur tourne sur la cible : quel budget CPU, que faites-vous s'il le dépasse ? » $\rightarrow$ S5-S6.
8. « Comment garantir que le code embarqué correspond au modèle ? » $\rightarrow$ back-to-back Python/C++ (S6).
9. « PID vs LQR : lequel choisir ? » $\rightarrow$ S6-S8, et pourquoi le LQR décroche loin du hover.
10. « Pourquoi un observateur plutôt qu'un intégrateur ? » $\rightarrow$ S7-S8, tes courbes.
11. « Comment organisez-vous une campagne d'essais ? » $\rightarrow$ plan de vol, météo, une modification par vol, métriques définies avant (S9).

6. Environnement de développement

- Ubuntu 24.04 natif, toolchain PX4 officielle, Gazebo Harmonic, QGC stable, VS Code, PlotJuggler, Flight Review.
- Python (numpy/scipy/matplotlib/pyulog/pytest, python-control pour le LQR) ; C++17, CMake, GoogleTest figé, `-Wall -Wextra -Wpedantic` sur `core/`.
- Versions figées dans `docs/versions.md` (PX4 v1.17.0 d6f12ad1, gtest 1.18.0, cmake 3.28.3, gcc 13.3.0, arm-none-eabi-gcc 13.2.1, QGC 5.1.4).
- Terrain : PC portable (QGC en USB pour paramètres, calibrations, prechecks, export), téléphone (météo, vidéo, chrono), carnet de plan de vol.
- Atelier : clés hex du kit, multimètre, rilsan, velcro, mousse 3M ; fer à souder uniquement pour les 4 fils du RP1.

7. Achats

7.1 Reçu le 12 septembre : Pixhawk 6C Mini (STM32H743, 2 IMU, header PWM intégré), GPS M10 (u-blox M10 + IST8310) → GPS1, Holybro H-Flow (PAA3905 + AFBR-S50 + IMU) → CAN.

7.2 Commande 2 — à passer entre le 15 et le 19 septembre, une fois la carte flashée et vérifiée (TP 0.4) ; au-delà du 19, le montage glisse en S2.

Poste	Référence	Prix ≈	Note
Plateforme	Holybro X500 V2 ARF (4× 2216, 4× ESC BLHeli_S 20 A, 6× hélices 1045, PDB, outils)	350 €	Commander là où c'est en stock (studioSPORT / RC Innovations)
Alimentation	Holybro PM02 V3 (XT60, en ligne batterie → PDB)	35 €	
Radio / récepteur	RadioMaster Pocket Crush ELRS + RP1 (CRSF sur TELEM1)	95 €	Pas l'ER4 (PWM, pas de série) ; 4 fils à souder
Accus radio	2× 18650	15 €	
LiPo ×2	4S 5200 mAh XT60	77 €	Contrôle à réception : éléments à ±0,02 V, pas de gonflement
Chargeur + sac	SkyRC B6AC Neo, sac LiPo	70 €	
Balise de signalement	Modèle homologué du commerce	50-100 €	Obligatoire ≥ 800 g ; identifiant déclaré dans AlphaTango
Terrain	Lunettes, paracorde, mousqueton, 2 piquets, velcro, rilsan	15 €	Attache phase 1
Total		710-770 €	Livraison ~5 jours, montage le week-end du 26-27

7.3 Plus tard : cellule de charge 5 kg + HX711 (~7 €, octobre, pour le TP 4.3 ; repli balance de cuisine) ; hélices de rechange sur crash.

Câblage cible du 6C Mini : POWER ← PM02 · GPS1 ← M10 · CAN ← H-Flow · TELEM1 ← RP1 (CRSF, RC_CRSF_PRT_CFG) · AUX1-4 ← fils signal + masse des 4 ESC (DSHOT ou PWM, jamais sur MAIN) · TELEM2 et GPS2 libres.

8. Protocole d'essais et sécurité (terrain)

8.1 Cadre réglementaire (fait, à tenir à jour)

Catégorie ouverte **A3** : ≥ 150 m des zones résidentielles, commerciales, industrielles et de loisirs, aucune personne non impliquée, VLOS, ≤ 120 m. Formation A1/A3 et enregistrement AlphaTango faits ; numéro UAS-FR sur la cellule ; **signalement électronique** ≥ 800 g : balise installée, identifiant déclaré, émission vérifiée avant le premier vol. Responsabilité civile aéromodélisme à vérifier (sinon FFAM).

8.2 Progression obligatoire

- **Phase 0 — banc, sans hélices** : flash, airframe, sens et ordre des moteurs par l'onglet Actuators (jamais par la sérigraphie), calibrations, H-Flow visible, RC et kill testés moteurs armés, failsafes, balise.
- **Phase 1 — attache sur piquets** : paracorde de 2-3 m sur un point structurel, lest ≥ 20 kg si sol meuble, hover ≥ 1,5 m. Sert aux premiers vols stock et au contrôle des vibrations — **pas aux lois de commande maison** : à l'étage position/vitesse, la corde fausse tout.
- **Phase 2 — vol libre bas** (2-5 m), opérateur à 10-15 m, pousse sur le kill, bascule stock/custom testée au sol. C'est ici que voit le jour toute brique maison.
- **Phase 3 — campagnes** (5-15 m, trajectoires de 10-40 m), quand les phases précédentes sont vertes sur la brique concernée.

Référence stock d'abord : autotune PX4 et vols Position/Altitude stock avant toute brique maison — baseline et mode de secours.

8.3 Failsafes — deux jeux de paramètres

Sans télémétrie, les failsafes remplacent tes yeux. Le LQ ELRS et le buzzer sont tes seuls retours en vol.

	Jeu attache (phase 1)	Jeu vol libre (phases 2-3)
Perte RC NAV_RCL_ACT	Land	RTL, RTL_RETURN_ALT = 10 m
Geofence GF_ACTION	Land	RTL ou Land
Batterie basse	Land	alerte puis Land (25 % / 15 %)
Perte de position COM_POSCTL_NAVL	à choisir (voir note)	à choisir (voir note)
Altitude max	5 m	30 m

Perte de position — rien n'est acquis. COM_POSCTL_NAVL = 0 : bascule en Altitude si la hauteur est valide, sinon Stabilized — le pilote reprend aux sticks ; = 1 : Land si la hauteur est valide, **sinon coupure des moteurs** (flight termination). Avec ton ESKF en source, ce sont tes drapeaux de validité qui décident : le choix se fait explicitement, se vérifie en SITL sous les deux jeux, et le pilote s'entraîne à la reprise en Altitude/Stabilized en stock avant tout vol en source. Un RTL en attache tire sur la corde à pleine puissance : le jeu « attache » ne contient aucun RTL. Les deux jeux sont versionnés dans `px4/params/` et le jeu chargé est noté dans chaque plan de vol. Chaque failsafe est testé en SITL puis en vol stock avant la première brique maison.

Test réel de perte RC : radio réellement éteinte, drone à 3 m en vol libre, opérateur à 15 m, batterie pleine, jeu « vol libre ». Pendant la perte, le kill de cette même radio **n'existe pas** : la reprise passe par le rallumage de la radio (lien rétabli en ~1 s) puis kill ou reprise au stick. Le test se fait une fois, en stock, après validation en SITL — jamais sur une brique maison.

8.4 La session de vol (une demi-journée)

- **Déclencheur** : fenêtre météo, jamais le calendrier.
- **Sac** (checklist imprimée) : drone + hélices vérifiées · 2 LiPo chargées la veille · radio et PC chargés, câble USB · microSD · lunettes · piquets, paracorde, mousqueton · trousse (clés, rilsan, velcro, hélices) · téléphone · **plan de vol écrit** (objectif de données, scénarios, ordre, jeu de paramètres) · sac LiPo.
- **Pré-vol** : hélices serrées, bon sens · tension · fix GNSS, satellites, hAcc · prechecks QGC · kill testé moteurs au sol · balise émettrice · log actif · export des paramètres · relevé météo (vent, rafales, température, source).
- **Pendant** : un scénario = un segment logué ; **une modification par vol** ; on vole pour les données, tous les scénarios prévus.
- **Après** : logs + params + météo dans `flight-tests/AAAA-MM-JJ/`, Flight Review le jour même (vibrations, saturations, innovations), entrée de journal.

8.5 Perturbations et conditions

- **Vent** par plage (< 10, 10-20, 20-30 km/h) d'après la station la plus proche **et**, sur les vols EKF2, l'estimation à bord (fusion de traînée EKF2, TP 3.8) — pas une mesure indépendante, dis-le. **Sur les vols où ton ESKF est en source, EKF2 est arrêté et il n'y a pas de vent estimé** : covariable = station + vols stock alternés dans la même session (avant/après), limites de comparaison écrites dans le rapport. Maintenir EKF2 en parallèle pour ce seul besoin coûterait de l'intégration et du CPU : non.
- **Masse ajoutée** : 200 g velcro à position marquée (≈ 15 %).
- **Échelons** de vitesse et de position à amplitudes fixées.
- **Perte GNSS en vol** : par paramètre (aide GNSS d'EKF2 coupée en Position flow), basse altitude, en stock d'abord.

- Le vent n'est pas reproductible : **≥ 3 répétitions par case**, médianes, vent logué comme covariable ; **vols de réglage et vols d'évaluation séparés**.

8.6 Zones secondaires et LiPo

Hangar couvert : option non acquise, rien du plan n'en dépend (s'il se confirme : vols par mauvais temps, GNSS réellement absent — hold-out sans masquage). Jardin (zone résidentielle) : banc et vérifications sans hélices uniquement. LiPo : charge surveillée dans le sac, stockage 3,8 V/élément, accus > 15 °C avant vol, pas de vol sous 0 °C.

9. Métriques

Domaine	Métriques
Estimation — simulation	RMSE position/vitesse/attitude vs vérité · NEES sur les états dont la vérité est connue · NIS avant gating : part sous la borne basse et part au-dessus de la borne haute, séparément, bornes à la dimension de la mesure · taux de rejet · comportement à la perte GNSS injectée
Estimation — vol, mode GNSS	Cohérence vs EKF2 (RMSE vitesse, écart position) · innovations blanches, NIS · convergence des biais · retard estimé vs réglé
Estimation — hold-out	Dérive de position à 30 s et 60 s sans GNSS (m, et % de la distance) : médiane et pire cas sur ≥ 10 fenêtres non chevauchantes, par type de segment · dérive de cap
Embarqué	Temps d'exécution moyen et max observé (pas une borne pire cas) par module (perf), échéances manquées, charge CPU, RAM/flash
Contrôle	RMSE de suivi · dépassement · temps d'établissement · $\int u^2$ · % temps en saturation · rejet (écart max, temps de récupération) — par plage de vent
Process	Tests et CI verts · reproductibilité (un script → les mêmes chiffres) · vols par session, scénarios réussis, anomalies suivies jusqu'à correction

10. Risques et plans B

Risque	Mitigation
Kit reçu tard	S0-S2 sont largement simulation ; montage et vols glissent d'un sprint ; Allan et rejeu bureau ne demandent que la carte
Météo d'hiver	Voler pour les données à chaque fenêtre ; logs chargés en S2-S3 ; le rejeu absorbe ; cinq semaines de tampon ; B-vol et C sont les jalons exposés
Pas de télémétrie	Failsafes testés, deux jeux de paramètres, geofence, enveloppe douce, bascule stock par interrupteur
Crash à 1,4 kg	Brique maison à l'étage position/vitesse (dérive, pas retournement) ; vol libre bas ; hélices de rechange ; stock par interrupteur
ESKF trop lourd sur le H7	Profilage tôt d'un petit problème (TP 3.9) ; fréquence de propagation réduite ; float ; repli : ESKF en parallèle seulement, validé en rejeu
Attitude ESKF fausse en source → boucles internes stock déstabilisées	A-logiciel validé ; vols en parallèle avec seuil de cohérence à bord avant le passage en source ; altitude basse, vols courts ; COM_POSCTL_NAVL choisi et pilote entraîné à la reprise
Bascule stock ≠ custom mal maîtrisée	Conçue avant d'être codée (TP 4.2), testée en SITL sous consigne non nulle, puis au sol
Flash 2 Mo juste (98 % en config default)	Retirer des modules dans boardconfig au TP 3.9
GNSS bruité, flow médiocre	hAcc vérifié avant vol, R adaptatif, gating ; herbe rase, jour, 2-5 m pour le repli
Candidatures qui accélèrent	Projet présentable dès A-logiciel (rejeu) et B-rejeu : README et rapport à chaque jalon
Dérive de périmètre	Jalons ; aucune extension (§11) tant que D n'est pas vert

11. Extensions hors plan

À définir en fin de projet, selon le temps et les postes rencontrés — les énoncés complets sont dans la v2.2 archivée. Rien ici ne conditionne un jalon.

- **MPC** (ex-TP 5.1-5.5) : formulation linéaire position/vitesse, prototype cvxpy, solveur embarqué (OSQP code-gen ou TinyMPC), profilage H7, offset-free avec l'ESO. Même étage que ton PID/LQR : la comparaison serait homogène. À décider en janvier.
- **Guidage** (ex-TP 6.1-6.4) : FSM mission, min-jerk, A* sur carte à zones interdites, benchmark RRT*. Faible priorité pour des postes navigation.
- **Boucles internes maison** (rate + attitude à la cadence gyro) : si tout est vert avant février.
- **Saison 2** : RTK F9P (erreur absolue, couplage serré), companion + caméra (VIO comme nouveau modèle de mesure, ROS 2), Remote ID européen si un scénario l'exige.

12. Références

Une seule source se lit intégralement et lentement (Solà) ; deux à moitié ; le reste par sections, la ligne « Lecture acquise » du TP sous les yeux. Budget lecture ~35 h, dont la moitié en S0-S3.

1. J. Solà — *Quaternion kinematics for the error-state Kalman filter* (arXiv:1711.02508). §1-3, §5, §6 stylo en main ; §4 pour l'intégration du quaternion ; annexes pour vérifier tes jacobiens.
2. Kok, Hol, Schön — *Using Inertial Sensors for Position and Orientation Estimation* (arXiv:1704.06053). Ch. 1-4 : modèles IMU, aiding, ESKF.
3. Woodman — *An introduction to inertial navigation* (UCAM-CL-TR-696). 37 pages, tout : strapdown, dérive, Allan.
4. Mahony, Kumar, Corke — *Multirotor Aerial Vehicles* (IEEE RAM 2012). 12 pages, tout : modélisation.
5. Mahony, Hamel, Pflimlin — *Nonlinear complementary filters on SO(3)* (IEEE TAC 2008). Section du filtre explicite, ~20 %.
6. Bouabdallah & Siegwart — *PID vs LQ Control Techniques Applied to an Indoor Micro Quadrotor* (IROS 2004). 6 pages, tout.
7. Gao — *Scaling and bandwidth-parameterization based controller tuning* (ACC 2003) : le réglage de l'ESO par bande passante. Han 2009, *From PID to ADRC* (IEEE TIE) en complément. Wang & Shen — survey observateurs de perturbation pour quadrotor (Actuators 2024) §2-3.
8. Bailey et al. — *Consistency of the EKF-SLAM algorithm* (IROS 2006). 6 pages : NEES, NIS, bornes χ^2 . 8b. Chen, Biggie, Ahmed, Julier, Heckman — *Kalman Filter Auto-tuning through Enforcing Chi-Squared Normalized Error Distributions with Bayesian Optimization* (arXiv:2306.07225). **§II-III seulement** : bornes basse et haute du NIS/NEES (pessimiste vs optimiste), et pourquoi la moyenne seule ne suffit pas — la variance d'un NIS bien réglé vaut $2 \cdot n_z$. L'auto-tuning du reste du papier est hors périmètre.
9. Åström & Murray — *Feedback Systems* (PDF Caltech). Ch. 11 (PID industriel), ch. 7 (retour d'état, LQR), ch. 3 (modélisation).
10. Honegger et al. — *An open source and open hardware embedded metric optical flow CMOS camera* (ICRA 2013) : dérivation flow → vitesse, pour le TP 3.2b.
11. Barfoot — *State Estimation for Robotics* (PDF libre) : ch. 6-7 quand tu bloques sur les rotations.
12. Beard & McLain — [mavsim_public](#) : ch. 4 (forces, vent Dryden), ch. 7 (GNSS, baro, mag).
13. Documentation opérationnelle, au moment du TP : PX4 v1.17 (Architectural Overview, uORB, Hello Sky, EKF2 dont retard GNSS et fusion de traînée, Multicopter control architecture, Failsafes, Geofence, Position mode, Autotune, DShot, DroneCAN, Optical flow, ulog, Failure injection, System startup) ; **le code** `src/modules/ekf2` (ring buffers,

horizon retardé) et `src/lib/matrix` ; GoogleTest primer ; docs Holybro ; **u-blox M10 integration manual et UBX-NAV-PVT** (hAcc/vAcc/sAcc, velNED, latence) ; datasheets PAA3905, AFBR-S50, ICM-42688-P, BMI055 ; wiki `allan_variance_ros`.

14. Réglementaire : arrêté du 27 décembre 2019 (signalement ≥ 800 g), notice de la balise, guide DGAC, [AlphaTango](#).
15. Vidéos gratuites, en primer avant la lecture : MathWorks Tech Talks *Understanding Kalman Filters* (avant le TP 2.3), *Understanding PID Control* (avant le 4.1), *State Space et Trimming and Linearization* (avant le 4.5).

Références des extensions (Mellinger, Rawlings-Mayne-Diehl, OSQP/TinyMPC, Karaman-Frazzoli, LaValle, Red Blob Games) : dans la v2.2 archivée.

Partie II — Parcours d'apprentissage (mode TP, par sprints)

La Partie I dit quoi et quand ; celle-ci dit comment apprendre. Aucune solution n'est donnée : le chemin, les vérifications, les pièges. Numérotation conservée de la v2 pour la continuité du journal ; retirés : 1.4 (Simulink), 5.x (MPC), 6.1-6.4 (guidage) ; nouveau : 3.9 ; 6.5 devient 7.1.

Mode d'emploi

Le contrat. Chaque TP : Objectif · Lire · Lecture acquise si tu sais · Faire · Valider · Piège. On ne passe au suivant que si « Valider » est vert. Le bilan de sprint (annexe A) décide Go/No-Go.

La règle d'apprentissage : lire la référence → redériver à la main → écrire un test minimal → implémenter → comparer à une référence. Jamais de copier-coller d'implémentation pour les briques cœur (ESKF, contrôleurs, observateur, dérivations). Exceptions assumées, ce sont des outils : firmware de la balise, banc HX711, scripts d'extraction ulog, CI, squelettes CMake.

Lire une référence en trois passes : survol de 10 minutes ; lecture ciblée avec la ligne « Lecture acquise » sous les yeux ; stylo en main sur les seules équations que le TP demande. Si après la passe 2 tu ne peux pas répondre à voix haute, relis — puis implémente : la compréhension finit de se faire en codant.

Attaquer une étape floue : quelle est l'entrée ? la sortie ? comment saurai-je que c'est juste ? Sans réponse à la troisième, écris le test d'abord.

Quand tu bloques : 1. relis la référence — 80 % des blocages sont une équation mal lue ; 2. cas minimal (1 axe, bruit nul, entrée constante) ; 3. les sept suspects de l'annexe C ; 4. doc PX4 et discuss.px4.io avec ton cas minimal ; 5. le tuteur, **avec ton hypothèse**.

Budget : les durées sont des estimations bureau ; ajoute ~4 h par sprint qui vole. **Journal** : une entrée par session (annexe A), trois minutes.

Vue des sprints

Sprint	Dates	TP dans l'ordre	Bureau	Jalon
S0	2 → 20 sept	0.1 · 0.2 · 1.1 · 1.2 · 0.4 · 1.3	~51 h	—
S1	21 sept → 4 oct	2.1 · 1.5 · 2.2 · 2.5a · 3.4 · 0.3	~48 h	—
S2	5 → 18 oct	2.3 · 2.4 · 3.7 · 3.5 · 2.5b	~57 h	A-logiciel · A-plateforme
S3	19 oct → 1er nov	3.0 · 3.1 · 3.2a	~40 h + Toronto	—
S4	2 → 15 nov	3.2b · 3.3 · 3.6 · 3.8	~38 h + Toronto	B-rejeu
S5	16 → 29 nov	3.9a · 3.9b · 4.1 · 4.3	~34 h	—
S6	30 nov → 13 déc	4.2 · 4.4	~28 h + vols	B-vol · C-PID
S7	14 → 27 déc	4.5 · 4.6a	~20 h	—
S8	28 déc → 10 jan	4.6b · LQR en vol	~14 h + vols	C
S9	11 → 24 jan	7.1	~26 h + vols	D

Sprint 0 — 2 → 20 septembre : fondations et carte (~51 h)

TP 0.1 — Logistique et fondations du dépôt (4 h) — fait

Objectif. Ne plus jamais être bloqué par la logistique.

Faire : dépôt avec l'arborescence du §3.2 ; CMake + GoogleTest, `ctest` vert ; PX4 cloné -- recursive, branche figée sur le tag stable, docs/versions.md ; **CI minimale** : un workflow GitHub Actions qui compile core/ et lance `ctest` à chaque push (outil, 30 min) ; commande 2 planifiée (§7.2).

Valider : `ctest` vert en local et en CI ; version PX4 notée ; date de commande dans le journal.

Piège : suivre main de PX4.

TP 0.2 — Environnement et premier vol SITL (8 h) — fait

Objectif. Un vol simulé complet avant toute théorie, sur le même modèle que ton drone.

Faire : toolchain PX4, QGC, Python ; `make px4_sitl gz_x500`, mission QGC de 3 waypoints ; uLog dans Flight Review et PlotJuggler (attitude, setpoints, sorties moteurs, `vehicle_gps_position`, `vehicle_local_position` et ses `ref_*`) ; `make px4_fmu-v6c_default` sans carte. Note ce que le SITL simule réellement (GNSS 30 Hz vs 5-10 Hz réel, capteurs disponibles, injections possibles).

Valider : mission réussie ; uLog annoté dans `flight-tests/sitl/` ; build cible OK.

TP 1.1 — Rotations et conventions, une fois pour toutes (10 h) — fait

Objectif. Éliminer d'avance la première cause de bugs de toute la suite.

Lire : Solà [1] §1-3 ; doc PX4 « Flight Controller Orientation ».

Lecture acquise si tu sais : tourner un vecteur avec un quaternion · dire dans quel sens va TA convention · pourquoi q et $-q$ codent la même rotation · actif vs passif.

Faire : papier (NED, FRD, lacet 90° , Hamilton $w-x-y-z$, « q_{nb} transforme du corps vers le monde » dans tes mots) ; `sim/rotations.py` (produit, conjugué, rotation, $q \rightleftharpoons R \rightleftharpoons \text{Euler}$, intégration sous ω constante) ; tests avant de te croire : $q \otimes q^* = 1$, $RR^T = I$, $i \otimes j = k$, aller-retour Euler, cas proches de 0° et de 180° , $q/-q$, erreur angulaire géométrique entre deux rotations.

Valider : tests verts ; `docs/conventions.md` figé.

Piège : Hamilton vs JPL ; $w-x-y-z$ vs $x-y-z-w$ (scipy) — ne jamais mélanger.

TP 1.2 — Dynamique du quadrotor, à la main (9 h) — en cours

Objectif. Posséder le modèle du simulateur et du contrôleur, avec les ordres de grandeur de ton drone.

Lire : Mahony-Kumar-Corke [4] en entier ; mavsim [12] ch. 4 pour les forces et le vent.

Lecture acquise si tu sais : écrire Newton-Euler de mémoire · d'où vient $\omega \times J\omega$ · pourquoi un quadrotor est sous-actionné · la traînée en fonction de la vitesse **air** (sol – vent), et la différence entre « par rapport à » et « exprimé dans ».

Faire :

1. Newton-Euler 6 DDL (fait) : translation (gravité, poussée, traînée rotor linéaire + fuselage quadratique en vitesse air), rotation ($\omega \times J\omega$). État en v_n (NED).
2. Actionneur : $f = k_f \Omega^2$, $m = k_m \Omega^2$, moteur du 1er ordre ; ordres de grandeur X500 (masse $\sim 1,4$ kg, hover $\sim 40-50\%$, inertie par cylindres et points masse) dans `docs/params-x500.md` avec source et incertitude « non mesuré ». Le banc du TP 4.3 remplacera k_f par une courbe commande $\rightarrow$ poussée : prévois l'interface.
3. Matrice d'allocation B, géométrie X, numérotation et sens PX4 — dessinée, affichée au mur.
4. Vent = vent moyen + rafales (Dryden simplifié ou bruit filtré), entrant par la traînée.

Valider : B donne un lacet nul à moteurs égaux, un roulis pur en déséquilibre gauche/droite ; un vent de face constant produit un équilibre incliné, calculé à la main.

Piège : la numérotation des moteurs diffère entre PX4, Betaflight et les papiers — la tienne au mur.

TP 0.4 — Réception de la carte : flash, M10/H-Flow, logs Allan (6 h)

Objectif. Vérifier le matériel avant la commande 2, et faire tourner la carte réelle pendant que tu finis la théorie : un log Allan dure 2-3 h et ne demande rien de toi. Outillage : va vite.

Lire : doc PX4 v1.17 « Pixhawk 6C Wiring Quick Start », « DroneCAN », « Logging » (`SDLOG_PROFILE`, `SDLOG_MODE`) ; pages Holybro du 6C Mini, du M10, du H-Flow.

Faire :

1. **Flash** : `make px4_fmu-v6c_default upload` sur ta branche figée, carte en USB seule. Console MAVLink dans QGC : `ver all, uorb top -1` (deux instances de `sensor_gyro` et `sensor_accel`), `listener` sur chacune — fréquences et températures dans le journal. `param export` → `px4/params/`, numéros de série dans `docs/versions.md`.
2. **M10 sur GPS1**, au bord d'une fenêtre ou dehors : `listener sensor_gps` (fix 3D, satellites, hAcc/vAcc/sAcc), `listener sensor_mag`. Logue 20-30 min statique avec fix : premier nuage GNSS réel, à ressortir au TP 3.2a (dispersion mesurée vs hAcc annoncé).
3. **H-Flow sur CAN** : `UAVCAN_ENABLE` en « Sensors Automatic Config » (valeur dans la doc v1.17), redémarrage, `uavcan status`, `listener sensor_optical_flow`, `listener distance_sensor` ; une feuille imprimée à 30 cm que tu déplaces. Note l'orientation par défaut du capteur.
4. **Logs Allan** : `SDLOG_MODE` en « from boot » (rien n'arme sans cadre), `SDLOG_PROFILE` avec « high rate » et « sensor comparison », carte sur mousse sur un meuble lourd, 15 min de chauffe puis 2-3 h statique ; un second log un autre jour. Copie dans `flight-tests/bench/` avec un `.md` (température, durée, support).
5. **Commande 2** dès que 1-3 sont verts, avant le 19.

Valider : deux IMU, M10 (fix, hAcc), mag et H-Flow (flow + distance) vus dans `listener` ; deux logs Allan ≥ 2 h lisibles par `ulog_info` ; commande passée.

Piège : loguer « armé seulement » et découvrir un log vide ; H-Flow sur un port UART parce que le connecteur rentre.

TP 1.3 — Simulateur Python (14 h)

Objectif. Ton banc de vérité terrain — extensible aux capteurs (TP 3.0) et à biais connus (NEES au TP 3.3).

Faire : RK4 générique testé sur l'oscillateur harmonique ; dynamique du TP 1.2 + moteurs 1er ordre + traînée + vent (nul par défaut), état $p, v, q, \omega, \Omega_{1..4}$; tests analytiques (chute libre, hover à $T = mg$, impulsion de moment, vent constant → dérive prédite à la main) ; architecture **vérité / capteurs séparés** : le TP 3.0 branchera des générateurs sur l'état sans toucher à la dynamique.

Valider : tests verts ; $\|q\| = 1$ à renormalisation près documentée.

Piège : quaternion intégré comme un vecteur ; pas de temps trop grand qui ressemble à un bug de modèle. Sans Simulink, ton simulateur n'a pas d'oracle externe : les cas analytiques et, plus tard, la comparaison qualitative au SITL en tiennent lieu.

Sprint 1 — 21 septembre → 4 octobre : IMU réelle, premier Kalman, drone assemblé (~48 h)

TP 2.1 — Caractérisation IMU par variance d'Allan (12 h)

Objectif. Mesurer le bruit de TES centrales — les chiffres qui rempliront Q.

Lire : Woodman [3] section Allan ; wiki `allan_variance_ros` (lecture des pentes) ; datasheets ICM-42688-P et BMI055.

Lecture acquise si tu sais : ce que représentent la pente $-1/2$ et le plancher · relier l'ARW (bruit continu) au terme de Q **après discrétisation** · distinguer instabilité de biais et marche aléatoire de biais · pourquoi le log doit durer $\sim 100\times$ le τ visé.

Faire : (étape 1, les logs, faite au TP 0.4) extraire gyro et accéléro bruts des deux IMU avec `pyulog` ; implémenter l'Allan overlapping toi-même, validé sur bruit blanc synthétique (pente $-1/2$) ; $\sigma(\tau)$ pour les 12 axes ; ARW/VRW et bias instability ; comparer aux datasheets et entre IMU, entre les deux jours.

Livrable : `docs/allan-report.md`.

Valider : courbe synthétique conforme ; ordres de grandeur réels cohérents avec la datasheet (facteur 2-3 accepté) ; répétabilité entre les deux logs commentée.

Piège : log trop court ; température non stabilisée ; le frigo qui vibre.

TP 1.5 — Autopsie de PX4 (10 h)

Objectif. Savoir lire le stack avant d'y écrire.

Lire : doc PX4 « Architectural Overview », « uORB Messaging », « Hello Sky », « Modules & Commands », page EKF2 (GNSS, horizon retardé, `EKF2_GPS_DELAY`), « Multicopter control architecture ».

Lecture acquise si tu sais : dessiner la chaîne de modules avec les topics · topic vs paramètre · pourquoi les modules tournent sur des work queues · pourquoi EKF2 fusionne « dans le passé » · qui publie `vehicle_attitude_setpoint` et à partir de quoi.

Faire : SITL : `uorb top -1`, `listener` sur attitude, local position, GPS, `param show EKF2*` — qui publie quoi, à quelle fréquence, avec quel retard déclaré ; Hello Sky jusqu'au bout, puis modifié (abonnement à `vehicle_attitude`, publication d'un topic custom) ; lecture guidée de `mc_pos_control` **et** de `mc_rate_control` (`Run()`, abonnements, publications, classe de calcul) : diagramme entrées/sorties de chacun, avec unités et autorité des sorties ; le module custom compilé pour SITL et `px4_fmu-v6c`, chargé sur la carte.

Valider : module custom sur SITL et sur cible ; chaîne `ekf2` → `pos` → `att` → `rate` → `allocator` dessinée de mémoire, avec le contrat de `mc_pos_control` (entrées `trajectory_setpoint` avec NaN, sorties attitude + poussée, limites) — c'est celui que ton TP 4.2 devra honorer.

Piège : modifier les modules stock — le tien à côté, le stock reste ta référence.

TP 2.2 — Du complémentaire au Kalman, en 1D puis sur SO(3) (15 h)

Objectif. L'échelle du bloc estimation : 1D gain fixe → 1D Kalman → 3D gain fixe (Mahony) → 3D Kalman (ESKF, TP 2.3). Chaque marche ne change qu'une chose.

Lire : Tech Talks *Understanding Kalman Filters* avant l'étape 3 ; Mahony-Hamel-Pflimlin [5], section du filtre explicite.

Lecture acquise si tu sais : expliquer avec les mains le terme de correction · les deux équations de prédiction et les trois de correction de mémoire · ce que fait le gain quand $R \rightarrow 0$ et $R \rightarrow \infty$ · pourquoi P baisse à chaque mesure.

Faire :

1. Générateur d'IMU synthétique dans le simulateur (première brique du TP 3.0) : l'accéléromètre mesure la **force spécifique** — piège n°1 de la navigation. Bruits et biais depuis ton rapport Allan.
2. Complémentaire 1 axe, gain fixe : gyro seul dérive, accéléro seul est bruité et biaisé par les accélérations. Note le gain qui marche.
3. **Kalman 1D** (angle, biais gyro), écrit de zéro en numpy. Quatre expériences : (a) K démarre haut et converge vers une constante — compare-la au gain de l'étape 2 : le complémentaire est un Kalman en régime permanent ; (b) $\text{tr}(P)$ se contracte à chaque mesure et regonfle entre deux (accéléro à 10 Hz, gyro à 100 Hz) ; (c) **mise à l'échelle conjointe** de Q , R et P_0 par un même facteur : K ne change pas ; Q ou R seul : K change — l'intuition « seul le rapport compte » ne vaut que pour ça, et Q est une matrice ; (d) R dix fois trop petit : filtre trop confiant, NIS hors des bornes χ^2 . Puis une demi-page : ce qui change quand l'angle devient une rotation 3D, et pourquoi on estimera une petite erreur plutôt que le quaternion.
4. Mahony sur SO(3) avec estimation du biais gyro. **Ce qui est identifiable** : avec l'accéléro seul, les composantes horizontales du biais ; la composante autour de la verticale ne l'est pas sans référence de cap — écris-le avant de tester.
5. Scénarios : statique bruité, rotation pure, translation accélérée.

Valider : statique $< 0,5^\circ$; biais horizontaux injectés retrouvés par les deux filtres, biais vertical non retrouvé et expliqué ; l'estimation penche en accélération soutenue et tu sais pourquoi.

Piège : oublier la gravité dans l'accéléro synthétique ; confondre P et la covariance d'innovation $HPH^T + R$ — c'est la seconde qui sert au NIS.

TP 2.5a — Squelette du rejeu (4 h)

Objectif. Avoir la chaîne de rejeu avant d'avoir un filtre à rejouer : pyulog → CSV horodatés (IMU brutes des deux instances, attitude EKF2 ; plus tard GNSS, baro, mag, flow, ToF) → exécutable C++ → CSV de sortie → notebook. Traiter dès maintenant les horodatages en μs , les trous et doublons, la configuration (quel log, quels paramètres). Logs bureau : immobile 5 min, rotations lentes, vives, parcours dans la pièce.

Valider : un log bureau traverse la chaîne avec un filtre bouchon (attitude EKF2 recopiée) ; dt, trous et fréquences reportés.

TP 3.4 — Montage du X500 et phase 0 (5 h, une après-midi — annexe F)

Objectif. Un drone prêt pour le banc électrique, sans soudure hors les 4 fils du RP1.

Lire : annexe F ; doc PX4 « Pixhawk 6C Wiring Quick Start », DShot ; docs Holybro.

Faire : dérouler l'annexe F phases A → D. **Pas d'hélices** avant la fin du TP 3.7. Charger le jeu de paramètres « attache » (§8.3).

Valider : phase 0 verte : les 4 moteurs tournent dans le bon sens et le bon ordre depuis Actuators ; H-Flow visible ; radio appairée, kill testé moteurs armés ; pesée dans `docs/mass-budget.md`.

Piège : ordre des moteurs d'après la sérigraphie ; PM02 à l'envers ; TX sur TX.

TP 0.3 — Balise de signallement électronique (2 h)

Faire : identifiant relevé et déclaré dans AlphaTango, numéro UAS-FR apposé ; balise fixée sur la plaque supérieure, antenne dégagée du carbone et loin du RP1 (2,4 GHz tous les deux) ; émission vérifiée dehors avec fix (application du fabricant) ; LQ radio inchangé balise active.

Valider : identifiant déclaré ; émission vérifiée ; sur la checklist pré-vol.

Sprint 2 — 5 → 18 octobre : ESKF d'attitude, failsafes, premiers vols (~57 h)

TP 2.3 — ESKF d'attitude : la dérivation (14 h)

Objectif. Le cœur théorique, à posséder au stylo : le Kalman 1D passé en 3D sur une rotation.

Lire : Solà [1] §5 en entier et §6.1-6.2, lentement, en refaisant chaque étape ; Kok-Hol-Schön [2] ch. 4 en seconde lecture.

Lecture acquise si tu sais : pourquoi l'erreur d'attitude a 3 composantes quand le quaternion en a 4 · raconter propagation → correction → injection → reset, et ce que le reset fait à P · ce que chaque terme de Q représente physiquement · quand la correction par la gravité est valide (accélérations de translation faibles devant g) et ce qu'on fait sinon.

Faire :

1. État nominal (q , b_g), état d'erreur ($\delta\theta$, δb_g), et pourquoi l'erreur est 3D.
2. Propagation de l'erreur : F , G , Q depuis Allan. Recette : dynamique vraie, dynamique estimée, soustrais, garde le premier ordre en δx .
3. Correction accéléromètre : $h(x)$, H , injection, **reset de covariance** (le jacobien du reset, Solà §6.3).
4. Vérification numérique de F et H par différences finies : écart relatif $< 1e-5$ — non négociable.

Livrable : dérivation manuscrite + `check_jacobians.py`.

Piège : convention de signe de $\delta\theta$ (gauche vs droite) — celle de Solà, écrite en gros dans le journal.

TP 2.4 — ESKF d'attitude en C++ : la lib `core/` (18 h)

Objectif. Ta première brique embarquable, couverte de tests, et le prototype Python qu'elle porte.

Lire : GoogleTest primer ; `src/lib/matrix` de PX4.

Faire : prototype Python d'abord (le vecteur B2B en sort) ; décision documentée sur la bibliothèque matricielle — conseil fort : `matrix` de PX4 (header-only, tailles fixes, zéro allocation, utilisable sur PC) ; `core/estimation/` : `predict(gyro, accel, dt)`, `update_accel(a)`, état et P accessibles, aucune dépendance PX4, aucune allocation ; tests en miroir : propagation vs solution exacte, jacobiens vs différences finies, parité avec le Python sur une trajectoire exportée ; **float vs double** mesuré (le H7 calculera en float) ; taille de pile et mémoire statique relevées.

Valider : parité Python $\approx$ C++ $< 1e-6$ en double, écart en float chiffré ; tests verts ; zéro `new/malloc` dans `core/`.

Piège : ordre des arguments de tes conventions matricielles.

TP 3.7 — Failsafes, geofence et relevé du terrain (7 h) — avant tout vol libre

Objectif. Sans télémetrie, les failsafes sont tes yeux : les tester avant de leur confier 1,4 kg.

Lire : doc PX4 « Safety Configuration », « Geofence », « Return Mode », « Failure injection » ; §8.2-8.3.

Lecture acquise si tu sais : qui a l'autorité en vol (le commander) · ce que fait RTL exactement · pourquoi le jeu « attache » ne contient aucun RTL · pourquoi le kill ne sert à rien pendant une perte RC.

Faire :

1. **Relevé minimal** : une sortie sur le terrain, trace GNSS (téléphone ou carte alimentée en USB) du périmètre utilisable et du contour des obstacles ; polygone dans `docs/terrain.md` ; geofence QGC = polygone rétréci de 10 m, versionnée.
2. Les deux jeux de paramètres du §8.3 dans `px4/params/`, avec un `README` qui dit quand charger lequel.
3. SITL : chaque failsafe déclenché un par un (`SYS_FAILURE_EN = 1`, `failure gps off`, `failure rc_signal off`, moteur dégradé, franchissement de la geofence), sous les deux jeux ; réaction notée.
4. Plan écrit du test réel de perte RC (§8.3) pour le TP 3.5.

Valider : chaque failsafe fait la bonne action en SITL sous chaque jeu ; geofence chargée ; paramètres dans l'export de la carte.

Piège : tester RTL ou geofence en attache.

TP 3.5 — Premiers vols stock et campagne de logs n°1 (10 h + logistique)

Objectif. Voler en sécurité et récolter la matière première de toute la suite.

Lire : §8 en entier ; doc PX4 « Position Mode », « Autotune ».

Faire, sur au moins deux sorties :

1. **Attache** (jeu « attache ») : hover Position (≥ 10 satellites, $hAcc < 2$ m), 2 m, 60 s ; petits déplacements. Spectre gyro dans Flight Review **avant** toute suite (vibrations = hélices, moteurs).
2. **Vol libre bas** (jeu « vol libre ») : hover, carré de 10 m, rotations ; puis le test de perte RC du TP 3.7, une fois.
3. **Autotune stock** (vent < 10 km/h, 5-8 m) : baseline et mode de secours ; gains exportés. Puis **échelons d'attitude en Stabilized** (petits, logués) : la réponse des boucles internes stock, à identifier au TP 4.1.
4. **Campagne de logs n°1**, un scénario par segment : hover 60 s · carré 20 m à 2 m/s · rotations · montée/descente 2 $\approx$ 10 m · legs à vitesse constante 3, 5, 8 m/s dans le lit du

vent (TP 3.8) · Position flow seul à 2-3 m (aide GNSS d'EKF2 coupée, en attache d'abord) — utile mais **hors jalon A**.

5. Par vol : ulog, paramètres, météo, jeu chargé ; Flight Review le soir.

Valider (A-plateforme) : Position GNSS tenu ; autotune fait ; failsafes testés ; ≥ 3 vols logués complets.

Piège : changer deux choses entre deux vols ; voler sous les arbres.

TP 2.5b — Rejeu et confrontation à EKF2 (8 h)

Objectif. Le geste professionnel : valider hors ligne sur données réelles.

Faire : ton ESKF d'attitude dans la chaîne du 2.5a ; logs bureau puis vols réels ; attitude superposée à EKF2, erreur, biais ; désaccords analysés — ton filtre accéléro-seul ne connaît pas le cap : leçon d'observabilité, pas un bug ; rapport court `flight-tests/rapport-attitude.md`.

Valider (A-logiciel) : RMSE roulis/tangage $< 1^\circ$ statique, $< 3^\circ$ en vol calme ; biais gyro stables, plausibles vs Allan ; README et figure du jalon publiés.

Piège : timestamps en μs — un dt mal converti donne un filtre « presque bon », le pire genre de bug.

Sprint 3 — 19 octobre → 1er novembre : navigation, acte 1 (~40 h + Toronto)

Coursera Toronto démarre (abonnement 19 oct → 19 nov) : modules 1-3 pendant S3, 4-5 et projet final pendant S4. Le projet final est un ES-EKF IMU + GNSS + LiDAR : c'est ton TP 3.2a en version guidée — fais-le avant de finir le tien.

TP 3.0 — Capteurs synthétiques (6 h)

Objectif. Sans capteurs idéaux, le rituel « capteur idéal d'abord » du 3.2 n'a pas d'entrée. Chaque générateur apparaît juste avant son modèle de mesure.

Lire : Kok-Hol-Schön [2] ch. 3 ; mavsim [12] ch. 7 ; datasheets PAA3905 / AFBR-S50.

Faire, dans `sim/sensors.py`, depuis l'état vrai, bruit et biais à zéro par défaut : GNSS (position NED locale + origine, vitesse NED, 5-10 Hz, **retard configurable**, hAcc/vAcc/sAcc, coupure sur intervalle, bras de levier) · baro (biais lent + bruit) · mag (champ du terrain tourné dans le corps + biais dur) · flow (projection d'un point du sol, deux termes : translation/hauteur et rotation propre ; qualité nulle au-delà de 30 m ou 40°) · ToF (hauteur/cos(tilt), saturé à 30 m) · **pertes et retards injectables** partout.

Valider : à vitesse nulle et rotation pure, le flow vaut la rotation ; immobile à plat, le ToF vaut la hauteur ; le GNSS retardé de 150 ms rend la position d'il y a 150 ms ; une trajectoire « huit » exportée, idéale puis bruitée.

Piège : repère capteur vs repère corps (orientation de montage) — suspect n°5.

TP 3.1 — ESKF complet : dérivation et extension C++ (16 h)

Objectif. Étendre proprement ce qui marche — sur papier, en Python, puis dans `core/`.

Lire : Woodman [3] (strapdown, dérive) ; Solà [1] §5 pour position, vitesse, biais accéléro ; Kok-Hol-Schön [2] ch. 3-4.

Lecture acquise si tu sais : écrire la mécanisation strapdown · pourquoi un biais accéléro et une petite erreur d'attitude se ressemblent vus des mesures · ce qui dérive avec l'IMU seule, à quelle vitesse · pourquoi l'accéléromètre n'est plus une mesure de gravité ici mais une entrée de propagation.

Faire : état nominal (p, v, q, b_g, b_a, b_baro : 17) et d'erreur (16) ; mécanisation strapdown ; F, G, Q complets, tous les jacobiens re-vérifiés numériquement ; observabilité par écrit (IMU seule : quoi dérive, pourquoi), vérifiée en lâchant le filtre sans correction ; puis **extension de core/estimation/ à l'état complet** (propagation strapdown, F, G, Q en C++, float et double, tests de parité avec le prototype Python, ~4 h) et relevé du coût par pas (tailles de matrices,

opérations) pour le TP 3.9.

Valider : jacobiens $< 1e-5$; dérive IMU seule mesurée cohérente avec la prédiction papier ; parité Python $\neq$ C++ de la propagation complète.

TP 3.2 — Modèles de mesure, un par un, capteur idéal d'abord (32 h : 3.2a en S3 ~18 h, 3.2b en S4 ~14 h)

Objectif. Le morceau noble : transformer des capteurs hétérogènes en corrections propres, et gérer deux modes.

Lire : Kok-Hol-Schön [2] ch. 4 ; u-blox M10 integration manual, UBX-NAV-PVT ; le code `ekf2` de PX4 (ring buffers, horizon retardé) ; pour 3.2b : Honegger [10], datasheets flow/ToF.

Lecture acquise si tu sais : pourquoi la vitesse GNSS aide plus les biais accéléro que la position · ce qu'un retard non compensé fait au filtre (biais proportionnel à la vitesse, et pire en rotation) · pourquoi le couplage serré exige des mesures brutes que le M10 ne fournit pas · pourquoi la route sol n'est pas le cap.

Le rituel, pour chaque capteur : (a) $h(x)$ et H sur papier ; (b) H vérifié numériquement ; (c) mesure idéale du TP 3.0 → convergence exacte ; (d) bruit puis biais ; (e) gating χ^2 et rejet d'une aberration injectée, **NIS calculé avant gating**, dépassements bas et haut comptés séparément, taux de rejet reporté ; (f) **porté en C++ dans `core/`** avec test de parité contre le Python — le filtre que rejouent les TP 3.6 et 3.9 est celui-là, pas le prototype.

3.2a — sous-jalons séparés, dans l'ordre :

1. **GNSS position, fusion nominale** : origine NED figée à l'armement, R adaptatif depuis $hAcc/vAcc$ (plancher, plafond).
2. **Bras de levier** : $h = p + R(q) \cdot l$; sur la vitesse, $h = v + R(q) \cdot (\omega \times l)$.
3. **GNSS vitesse NED** : R depuis $sAcc$; c'est elle qui fixe b_a .
4. **Retard** : tampon circulaire d'états nominaux **et de covariance** horodatés ; la mesure est fusionnée contre l'état à son instant de validité, la correction est propagée au présent (relis `ekf2`). L'approximation « appliquer la correction telle quelle » se chiffre d'abord : valable à faible vitesse et sans rotation rapide, pas au-delà.
5. **Baro** : biais dans l'état ; souffle des hélices à regarder sur un log réel (hover vs sol).
6. **Mag** : déclinaison ; alignement initial ; en déplacement, cohérence attitude/vitesse GNSS — pas la route sol.
7. NIS par mesure, tracés, cohérence χ^2 ; premier nuage GNSS réel du TP 0.4 : dispersion vs $hAcc$.

Valider 3.2a : rituel passé pour chaque item ; en simulation mode GNSS, NIS dans les bornes entre ~90 et 98 % du temps, RMSE position < 1 m, vitesse $< 0,15$ m/s avec bruits réalistes ; le retard non compensé produit l'erreur prédite, la compensation la supprime.

3.2b (S4) : 8. Flow : rotation propre retirée, échelle par la hauteur sol **estimée** ; en mode GNSS, innovation surveillée non fusionnée ; en repli, mesure de vitesse. Qualité et incertitude formalisées. 9. **ToF** : distance le long de l'axe, inclinaison compensée, sol plat ; cas limite à 45° . 10. **Bascule de mode** : perte de fix ou gating répété → GNSS-denied sans reset de P ; retour → réinitialisation contrôlée de la position (saut documenté, covariance gonflée), hystérésis ; **effet du saut sur le contrôleur** écrit noir sur blanc (le TP 4.2 en aura besoin). 11. Pertes de flow (texture, altitude) : gating, dégradation vers IMU + baro/ToF.

Valider 3.2b : rituel passé ; coupure GNSS de 30 s sur la « huit » : dérive < 2 m, pas de divergence, retour propre.

Pièges : signe de la compensation de rotation du flow ; horodater le GNSS à la réception au lieu de la validité ; bras de levier dans le mauvais repère.

Sprint 4 — 2 → 15 novembre : navigation, acte 2 (~38 h + Toronto)

TP 3.2b (ci-dessus, ~14 h) puis :

TP 3.3 — Validation avec vérité : NEES en Python, puis SITL (10 h)

Objectif. Mesurer une erreur, pas une cohérence — là où la vérité existe.

Lire : Bailey et al. [8] : NEES, NIS, bornes χ^2 .

Lecture acquise si tu sais : dimensions et seuils χ^2 de tes NIS et NEES · pourquoi des échantillons corrélés faussent le comptage · ce qu'un NEES trop petit signifie.

Faire :

1. **Simulateur Python à biais connus :** NEES sur tous les états, y compris les biais ; introduis des écarts entre le modèle qui génère les données et celui du filtre (bruit non blanc, biais qui dérive) pour éviter une validation trop favorable.
2. **SITL x500 :** capteurs bruts et topics `*_groundtruth` logués ; rejeu de ton ESKF ; RMSE, NIS, NEES sur le sous-état dont la vérité est connue (position, vitesse, attitude).
3. Trajectoires variées (hover, carré 50 m, huit, montée) ; `failure gps off 30 s` et `60 s` : bascule, dérive, retour.

Valider : B-rejeu, volet simulation (§4.3).

TP 3.6 — Ton ESKF contre EKF2 sur TES vols, et le hold-out (9 h)

Objectif. Le livrable phare du volet estimation, avec une vérité terrain honnête.

Faire : vols réels rejoués en mode GNSS (positions, vitesses, innovations, NIS, biais, retard estimé vs réglé) ; **hold-out** `replay/holdout.py` : fenêtres de 30 s et 60 s **non chevauchantes** sur des vols réservés à l'évaluation, ≥ 10 par type de segment (hover, translation, virage), dérive en fin de fenêtre vs EKF2+GNSS, médiane et pire cas ; **échecs et limites publiés** à côté des médianes ; `flight-tests/rapport-estimation-v1.md` relisible par un tiers (méthode, courbes, écarts expliqués, cohérence $\neq$ erreur absolue).

Valider : B-rejeu, volet réel ; README, figure, commande de reproduction, tag.

Piège : comparer à EKF2 comme si c'était la vérité — écris la phrase inverse dans le rapport.

TP 3.8 — Terrain : carte et vent (5 h)

Objectif. Un repère fixe du terrain et une estimation du vent à bord, comme covariable des campagnes.

Lire : mavsim [12] ch. 4 ; doc PX4 EKF2, fusion de traînée (`EKF2_DRAG_CTRL`, `EKF2_BC0EF_*`, `EKF2_MC0EF`).

Faire : origine fixe du terrain (lat/lon) et polygones dans `docs/terrain.md` — distincte de l'origine locale que PX4 refixe à chaque armement (`docs/frames.md`) ; **coefficients de traînée** sur les legs à vitesse constante du 3.5, en air calme : en vitesse stabilisée l'accélération cinématique est nulle, la traînée se lit dans la **force spécifique en axes corps** (accéléro) contre la vitesse air — c'est ce que fusionne EKF2 ; renseigner `EKF2_BC0EF_X/Y`, `EKF2_MC0EF`, activer, comparer le topic `wind` à la station sur les vols suivants (direction et ordre de grandeur, sans seuil : la station est loin).

Valider : origine et polygones versionnés ; vent estimé cohérent en direction sur deux vols.

Piège : régresser la traînée un jour de vent — le vent devient le coefficient.

Sprint 5 — 16 → 29 novembre : ESKF sur cible, synthèse du PID (~34 h)

TP 3.9 — ESKF sur cible (18 h : 3.9a interface et parallèle ~8 h, 3.9b source et boucle fermée ~10 h) — nouveau v3

Objectif. Passer d'« un estimateur validé en rejeu » à « un estimateur qui pilote le véhicule ». C'est le TP qui manquait à la v2, et le principal risque technique du projet : le filtre C++ complet existe déjà (TP 3.1-3.2), ici on ne fait que l'interface, le profilage et la qualification.

Lire : doc PX4 « System Startup », « Modules & Commands », le message `vehicle_local_position.msg` (champs de validité, `ref_*`, compteurs de reset) ; `src/modules/ekf2` : ce qu'il publie et pourquoi.

Lecture acquise si tu sais : quels topics le commander et `mc_pos_control` consomment, et quels drapeaux de validité déclenchent le failsafe « perte de position » · ce qu'un reset de position doit signaler au contrôleur · pourquoi un maximum observé n'est pas une borne pire cas · **ce qu'une attitude fausse publiée par ton module fait aux boucles internes stock** — elles restent du code PX4, mais elles asservissent ton estimé.

3.9a — interface et parallèle :

1. **Wrapper** `px4/eskf/` : abonnements IMU (une instance, cadence choisie), GNSS, baro, mag, flow, ToF ; horodatage à la validité ; en **mode parallèle**, publication sur des topics custom loggés, EKF2 restant la source. Aucun calcul dans le wrapper : tout est dans `core/`.
2. **Profilage sur cible** dès le premier jour : `perf`, temps moyen et max observé de la propagation et de chaque correction, échéances manquées, pile, RAM, flash (retirer des modules dans `boardconfig` si nécessaire). Fréquence de propagation choisie d'après les chiffres, pas d'après l'envie.
3. Vols en parallèle (jeu « vol libre », stock) : ton estimé et EKF2 logués ensemble ; comparaison en rejeu **et** à bord (les deux doivent coïncider — sinon c'est l'horodatage ou le float). **Condition de passage en source** fixée à l'avance : écart d'attitude bord ESKF/EKF2 sous un seuil (par exemple $< 2^\circ$ RMS) sur ≥ 3 vols, position et vitesse cohérentes.

3.9b — source et boucle fermée :

4. **En source** : ton module publie `vehicle_local_position`, `vehicle_attitude` (et global position) avec drapeaux de validité honnêtes, compteurs et deltas de reset renseignés à chaque bascule de mode, `ref_*` figés à l'armement ; EKF2 arrêté par le script de démarrage. **La bascule EKF2 $\neq$ ESKF se fait au sol** (script) ; en vol, le filet est le failsafe perte de position (action de `COM_POSCTL_NAVL` choisie et vérifiée en SITL sur tes drapeaux, §8.3), l'altitude basse, le kill et un pilote entraîné à reprendre en Altitude/Stabilized — la bascule d'estimateur en vol est une extension. En source, EKF2 est arrêté : pas de vent estimé à bord sur ces vols (§8.5).
5. SITL en boucle fermée sur ton ESKF : hover et carré en Position, perte GNSS injectée (le contrôleur suit le repli, puis le saut du retour).
6. Vol en source (jeu « vol libre », 2-5 m, vols courts) : hover, carré, sur le stock d'abord (TP 4.4 viendra pour ton module).

Valider (B-vol, avec le TP 4.4) : chiffres de profilage dans `docs/eskf-target.md` ; parité bord/rejeu ; condition de passage en source atteinte et documentée ; hover et carré tenus en source ; failsafe perte de position vérifié en SITL sur tes drapeaux, sous l'action choisie.

Piège : publier une validité optimiste pour « que ça vole » — le commander te croit sur parole.

TP 4.1 — Synthèse de la boucle position/vitesse (10 h — Python)

Objectif. Concevoir avant de coder, sur un modèle qui inclut ce que tu ne remplaces pas : les boucles internes stock.

Lire : Tech Talks *Understanding PID Control* ; Åström & Murray [9] ch. 11 ; doc PX4 « Multicopter control architecture » et la conversion accélération → attitude + poussée de `mc_pos_control` ; tes cours d'asservissement.

Lecture acquise si tu sais : les trois ingrédients qui séparent un PID industriel d'un PID de TP · expliquer le windup avec la saturation d'inclinaison et de poussée · pourquoi la boucle externe doit être 5-10× plus lente que l'interne.

Faire :

1. **Identifier la boucle interne stock** sur les échelons d'attitude du TP 3.5 : consigne →

attitude, ajustée par un 2e ordre ou un 1er ordre + retard (bande passante, retard). C'est le cas industriel « boucle interne fournie ».

2. **Plant réduit** : translation du TP 1.2 + modèle de boucle interne + poussée (gaz de hover mesurés, courbe du TP 4.3 quand elle existe) + traînée + vent.
3. **Cascade** : position (P) → vitesse (PID : D filtré, anti-windup par back-calculation, feedforward de gravité et d'accélération) → accélération de consigne → **conversion en attitude + poussée** identique à celle de PX4 (même formule, mêmes limites). Réglage position après vitesse.
4. Robustesse : masse $\pm 30\%$, retard, vent 5 m/s + rafales, saturations (inclinaison max, poussée min/max), bruit de vitesse estimée réaliste.
5. **Vecteurs back-to-back** en CSV (entrées, sorties attendues, plusieurs séquences), versionnés dans `sim/vectors/`. Gains comparés en ordre de grandeur aux `MPC_XY_*` / `MPC_Z_*` du stock : un écart d'un facteur 10 signale un plant faux.

Valider : cascade stable sur toute la campagne ; vecteurs exportés et relus par un script indépendant ; `sim/README.md` (structure, gains, hypothèses).

Piège : régler la position avant la vitesse ; ignorer la limite d'inclinaison dans le prototype puis la découvrir en vol.

TP 4.3 — Banc de poussée (6 h)

Objectif. Une courbe commande → poussée mesurée, par moteur, avec des précautions de 10".

Faire : cellule 5 kg + HX711 + n'importe quel microcontrôleur (tutoriel externe autorisé : c'est un outil) ; support qui tient **le bras complet** (moteur + ESC) verticalement au-dessus de la cellule, aval dégagé, grillage entre toi et l'hélice, lunettes, personne dans le plan de rotation ; repli 0 € : bras sur levier et balance de cuisine ; calibration par masses connues ; paliers stabilisés par moteur, DShot ou PWM selon ta config. **Sans mesure de régime, tu identifies commande → poussée, pas $k_f(\Omega)$** : c'est ce que ton contrôleur utilise. Constante de temps seulement si la chaîne de mesure a la bande passante (sinon, dis-le et passe). Courbe injectée dans le simulateur et le plant du 4.1.

Valider : dispersion inter-moteurs quantifiée ; hover théorique (masse pesée / 4) sur ta courbe à $\pm 10\%$.

Piège : effet de sol du banc ; support qui résonne avec l'hélice.

Sprint 6 — 30 novembre → 13 décembre : ton module vole (~28 h + vols)

TP 4.2 — Ton module position/vitesse dans PX4 (16 h)

Objectif. Ton code pilote le drone (simulé d'abord), avec une reprise stock conçue avant d'être codée.

Lire : tes notes du TP 1.5 sur `mc_pos_control` ; doc PX4 « System Startup » ; `vehicle_attitude_setpoint.msg`, `trajectory_setpoint.msg`, `vehicle_constraints.msg`.

Lecture acquise si tu sais : le contrat complet de `mc_pos_control` (entrées avec NaN, sorties, limites, décollage et atterrissage) · ce qu'est un transfert sans à-coup et pourquoi il faut synchroniser les intégrateurs · pourquoi une seule autorité de publication.

Faire :

1. `core/control/` : cascade du 4.1, portable, testée ; **back-to-back** sur les CSV du 4.1, écart < tolérance numérique — ta preuve modèle $\equiv$ code.
2. **Wrapper** `px4/pos_control/` : mêmes abonnements et publications que `mc_pos_control`, mêmes limites, même conversion accélération → attitude + poussée, gestion des setpoints partiels, rampe de décollage. Consomme `vehicle_local_position` — donc ton ESKF

quand il est en source, sans changement de code.

3. **Reprise stock, conçue sur papier avant** : une seule autorité de publication ; deux instances qui calculent, une qui publie ; à la bascule, intégrateurs et états du module qui reprend initialisés depuis la sortie courante (transfert sans à-coup) ; interrupteur radio lu par les deux ; données fraîches vérifiées (timeout). Documentée dans `docs/switch.md`.
4. SITL x500 sur ton module : hover, carré, mission QGC ; **bascule stock $\neq$ custom testée sous consigne non nulle**, dans les deux sens, sans à-coup visible ; puis la même chose sur ton ESKF en source.
5. Sur cible : compilé, chargé, temps d'exécution mesuré, bascule testée au sol moteurs désarmés (sorties loguées).

Valider : back-to-back vert ; hover, carré, mission SITL ; bascule maîtrisée dans les deux sens sous mouvement ; chiffres sur cible.

Piège : tester dans un mode où tes setpoints n'arrivent pas (`listener` pour savoir qui publie) ; coder la bascule sans l'avoir dessinée.

TP 4.4 — Ton PID en vol (12 h + logistique)

Objectif. Le moment de vérité, en vol libre bas, une modification par vol.

Faire :

1. Gains initiaux : ceux du 4.1 rapprochés des gains stock (ordre de grandeur), jeu « vol libre », 2-5 m, opérateur à 15 m, bascule stock testée au sol le jour même.
2. Sur EKF2 : hover, échelons de vitesse, carré de 10 m ; réglage par vol (un gain à la fois), spectre et saturations vérifiés le soir.
3. **Sur ton ESKF en source** (TP 3.9), mêmes scénarios, PID figé : c'est B-vol.
4. Campagne comparative **stock vs ton module**, mêmes boucles internes, mêmes scénarios (hover 60 s, carré 20 m, échelons de vitesse), vent < 15 km/h logué, **2 scénarios $\times$ 3 répétitions**, script de métriques (RMSE, dépassement, temps d'établissement, $\int u^2$, % saturation) par plage de vent.

Valider (C-PID) : ton module vole sur EKF2 et sur ton ESKF ; performances comparables au stock ($\pm 20\%$), chiffres à l'appui ; README et figure du jalon.

Piège : tout changer entre deux vols ; « battre le stock » n'est pas le critère — un résultat robuste et expliqué l'est.

Sprint 7 — 14 → 27 décembre : LQR et ESO en simulation (~20 h)

TP 4.5 — LQR, puis LQI (12 h — Python, vol opportuniste)

Objectif. Du réglage par boucles au réglage par pondérations, au même étage que ton PID.

Lire : Tech Talks *State Space, Trimming and Linearization* ; Bouabdallah & Siegwart [6] en entier ; Åström & Murray [9] ch. 7.

Lecture acquise si tu sais : résumer leurs conclusions PID vs LQ et ce que tu prédis de différent sur un X500 · pourquoi le LQR décroche loin du hover · ce que l'action intégrale ajoute et ce qu'elle coûte.

Faire :

1. Linéariser la translation autour du hover avec le modèle de boucle interne du 4.1 : A, B sur papier, vérifiés par différences finies sur ton simulateur ; discrétiser à la cadence du module.
2. K hors ligne (`python-control`), pondérations façon Bryson ; itérer en simulation ; **casser la séparation d'échelles** exprès (Q trop agressif) pour voir l'interne décrocher.
3. LQI pour l'erreur statique (masse, vent) ; comparaison PID / LQR / LQI sur les mêmes scénarios que le 4.1.

4. `core/` : une matrice de gain dans le même wrapper que le PID — la difficulté est l'interface (vecteur d'erreur, consignes, saturations, anti-windup de l'intégrateur).
5. SITL ; **en vol si une session le permet** (un vol, hover et carré, sans campagne).
6. Demi-page « LQG et séparation » dans le journal, hypothèses explicites.

Valider : LQR et LQI stables sur toute la campagne de simulation ; comparaison documentée ; SITL vert.

Piège : comparer des lois réglées dans des conditions différentes.

TP 4.6a — ESO : l'observateur, en simulation (8 h)

Objectif. Le rejet de perturbation en protocole propre — ton terrain de PFE, sur la vraie perturbation. La question à laquelle tu répondras : *pourquoi un observateur plutôt qu'un intégrateur ?*

Lire : Gao 2003 [7] (réglage par bande passante) ; Han 2009 en complément ; Wang & Shen §2-3.

Lecture acquise si tu sais : l'idée ADRC en une phrase · pourquoi un ESO linéaire est un observateur de Luenberger sur le modèle augmenté d'un état de perturbation · pourquoi la bande passante est un compromis vitesse/bruit · ce que l'ESO suppose connu et ce qu'il ne suppose pas.

Faire :

1. ESO linéaire sur la boucle de vitesse (état étendu = force de perturbation par axe), gains par bande passante ; `core/`, testé sur perturbations synthétiques (échelon de force, sinus, rafales Dryden) : erreur d'estimation et retard en fonction de la bande passante, **avec le bruit et le retard de vitesse des deux estimateurs** (EKF2 et ton ESKF, mesurés au 3.6) : la bande passante retenue doit tenir pour les deux, sinon note deux réglages et dis-le.
2. Compensation en feedforward dans la cascade du 4.1. Quatre configurations en simulation : PID · PID + ESO · LQI · LQR + ESO, scénarios {échelon de vent 5 m/s, rafales, masse +15 %, échelon de vitesse}.
3. Ce que chacun suppose : l'intégrateur suppose une perturbation lente ; l'ESO suppose un modèle nominal et une bande passante ; écris-le, avec les cas où l'un bat l'autre.

Valider : courbes écart max / temps de récupération / sensibilité au bruit en fonction de la bande passante ; choix de la bande passante pour le vol argumenté.

Piège : bande passante trop haute = bruit qui « ressemble » à de la performance en simulation et détruit tout en vol.

Sprint 8 — 28 décembre → 10 janvier : rejet de perturbation en vol (~14 h + vols)

TP 4.6b — PID vs PID + ESO en vol (6 h + vols)

Faire : PID figé (TP 4.4), **sur EKF2** (l'estimateur n'est pas la variable, et le vent estimé à bord reste disponible), jeu « vol libre » ; deux configurations (sans / avec ESO) × scénarios {hover en vent 10-20 km/h, masse ajoutée 200 g, échelon de vitesse}, **3 répétitions**, médianes, vent estimé à bord comme covariable ; métriques par script ; LQR en vol si une session le permet (TP 4.5). Ce sprint absorbe aussi ce que Noël a laissé.

Valider (C) : comparaison reproductible à vent comparable — écart max, temps de récupération, sensibilité au bruit, J_{u^2} — et **décision écrite** : ESO retenu ou écarté pour la mission finale, avec les courbes qui la justifient. Un PID bien réglé déjà suffisant, ou un ESO qui gagne en rejet et perd en bruit, sont des résultats aussi valables qu'une amélioration nette ; expliquer pourquoi une solution n'a pas été retenue est une preuve de jugement.

Piège : conclure sur une répétition ; comparer à vents différents.

Sprint 9 — 11 → 24 janvier : campagne cumulative et valorisation (~26 h + vols)

TP 7.1 — Campagne cumulative, rapport, README (26 h + vols)

Objectif. Transformer cinq mois en preuves consultables en dix minutes — sans campagne factorielle.

Faire :

0. **Qualifier la configuration retenue sur ton ESKF** (~2 h + un vol court) : SITL en boucle fermée sur ton ESKF avec le contrôleur du jalon C (hover, carré, échelon de vent injecté, perte GNSS de 30 s), puis un vol court en source (hover, carré, échelon de vitesse, jeu « vol libre »). Si l'ESO doit être re-réglé pour tenir sur ton ESKF, c'est une configuration distincte : la noter et la justifier — pas une campagne de plus. Rien ne se gèle avant.
1. Geler le code (tag). Suite cumulative : (a) corpus de rejeu estimation (TP 3.6, complété des vols récents) ; (b) stock vs ton PID, mêmes boucles internes (TP 4.4) ; (c) PID vs PID + ESO (TP 4.6b) ; (d) **mission QGC standard** (décollage → 3-4 waypoints → atterrissage) volée 3 fois sur ton ESKF en source et le contrôleur retenu au jalon C, et 3 fois en stock, mêmes conditions ; sans vent estimé à bord sur les vols ESKF (§8.5), les vols stock encadrent les vols ESKF dans la même session. Une seule sortie suffit si la météo le permet ; sinon deux.
2. Rapport type *flight test report* (8-12 pages) : plateforme, architecture, méthodes, résultats chiffrés, **limites honnêtes** (cohérence vs erreur absolue, vent non reproductible, boucles internes stock), anomalies et corrections, suites (§11).
3. README : courbes clés, GIF, comment reproduire chaque figure ; une vidéo courte ; un post technique (l'ESKF INS/GNSS en source par le rejeu et le hold-out).
4. CV et LinkedIn avec le pitch du §1, tes chiffres, et le **niveau atteint pour chaque brique : simulation, rejeu réel, sur cible, en vol.**

Valider : D ; un ingénieur extérieur comprend le projet en dix minutes de README.

Après le 24 janvier : tampon météo et rattrapage jusqu'au 28 février ; extensions du §11 seulement si D est vert.

Cours, en parallèle des TP

Un seul cours à la fois, au moment du TP qu'il recouvre ; il remplace l'essentiel des heures de lecture de ce TP, jamais le travail ni la validation.

Quand	Formation	Recouvre	Coût
S1, S5, S7	MathWorks Tech Talks <i>Understanding Kalman Filters</i> (avant le 2.3), <i>Understanding PID Control</i> (avant le 4.1), <i>State Space + Trimming and Linearization</i> (avant le 4.5)	Primers vidéo d'une à deux heures	0 €
S3-S4 (19 oct → 19 nov)	Coursera — <i>State Estimation and Localization for Self-Driving Cars</i> (U. Toronto, ~27 h) : modules 1-3 en S3, 4-5 + projet final (ES-EKF IMU + GNSS + LiDAR) en S4	TP 3.1, 3.2	un mois d'abonnement ; téléchargement notes et solutions avant résiliation
Fait	Formation A1/A3 (AlphaTango)	Réglementaire	0 €

Pour un recruteur GNC, ton dépôt et tes rapports d'essais pèsent plus que n'importe quel certificat.

Annexes

A. Gabarits de journal

Session : ## AAAA-MM-JJ – Sx – TP a.b étape n titre – durée (prévu) puis : **Objectif** · **Fait** ·

Bloqué sur (avec l'hypothèse) · **Décision** · **Prochaine session** · **Jokers**.

Bilan de sprint (30 min, dernier jour) : heures réelles vs prévues (logistique de vol comprise) · TP verts / rouges · jalon Go ou No-Go · ce qui glisse et où · météo des deux semaines, vols réalisés · décision pour le sprint suivant · calendrier décalé ou ravané.

B. Gabarit de mini-rapport d'essai (un par campagne)

Objectif · Configuration (versions, tag, gains, jeu de paramètres, masse) · Conditions (date, heure, vent station et vent estimé à bord, température, lumière, satellites, hAcc) · Scénarios et répétitions · Métriques (tableau, par plage de vent) · Figures · Interprétation · Anomalies et suites · Conclusion.

C. Les sept suspects universels (à chaque bug de nav ou de contrôle)

1. Convention de quaternion, ordre des composantes.
2. Signe et repère de la gravité — force spécifique.
3. Unités (deg/rad, g/m·s⁻², lat/lon en 1e-7 deg dans les messages GNSS).
4. Timestamps et dt (µs vs s).
5. Repère capteur vs corps (orientation de montage du H-Flow, du M10).
6. Retard et horodatage des mesures : une mesure GNSS vaut à l'instant de fixation, pas de réception — 150 ms à 5 m/s = 75 cm.
7. Origines et bras de levier : origine locale PX4 (refixée à l'armement) vs origine fixe du terrain ; antenne vs IMU.

F. Guide de montage X500 V2 (une après-midi, à ouvrir à réception)

~1 h de mécanique, ~1 h de câblage, ~2 h de configuration. Outils du kit ; multimètre, rilsan, velcro, mousse 3M.

F.1 — La veille : inventaire et pesée de chaque pièce → docs/mass-budget.md ; accus en stockage ; notice Holybro X500 V2 et « Pixhawk 6C Wiring Quick Start » lues.

F.2 — Phase A, mécanique (hélices interdites jusqu'à la phase E)

1. Plaques, bras (moteurs et ESC pré-installés), train, plateau batterie ; frein-filet bleu sur les vis structurelles.
2. Repérer l'avant ; FC au centre de la plaque supérieure sur mousse 3M épaisse + strap, **flèche vers l'avant**, loin des fils de puissance.
3. Maquette avant fixation : M10 au plus haut, loin de la PDB et des ESC ; RP1 à l'arrière, antenne dégagée du carbone ; balise loin du RP1 (2,4 GHz tous deux) ; H-Flow **sous la plaque inférieure**, objectifs dégagés, connecteurs vers l'arrière (orientation par défaut Holybro, sinon paramètre de rotation).

F.3 — Phase B, puissance 4. XT30 des bras dans les XT30 de la PDB. 5. PM02 V3 en ligne : XT60 batterie → entrée PM02 → sortie PM02 → XT60 de la PDB ; câble 6 fils → POWER.

Multimètre : pas de court + / – sur l'entrée batterie avant tout branchement.

F.4 — Phase C, signal 6. Fils servo des 4 ESC → AUX 1-4 (signal + masse ; fil rouge non connecté). Jamais sur MAIN. 7. M10 → GPS1 ; H-Flow → CAN ; RP1 → TELEM1 (**TX ≠ RX croisés**, 5 V, GND) sur câble JST-GH 6 broches — les 4 fils soudés sur les pads du RP1 : la seule soudure du projet. 8. Cheminement loin de la puissance, colliers souples ; microSD dans le FC.

F.5 — Phase D, premier allumage (sans hélices) 9. USB seul : QGC voit le FC, PX4 figé déjà flashé au TP 0.4 ; airframe « Holybro X500 V2 » si présent, sinon quad X générique. 10. Batterie (smoke test 2 s, débrancher, sentir, rebrancher) : ESC qui chantent, tension et courant cohérents (calibration PM02 selon Holybro). 11. Calibrations : accéléro, gyro, boussole (dehors, loin des voitures), niveau. DroneCAN déjà vérifié carte seule au TP 0.4 : contrôle en cinq minutes. 12. Radio : binding phrase identique Pocket ≠ RP1, RC_CRSF_PRT_CFG = TELEM1,

voies vérifiées ; kill switch et interrupteur stock/custom assignés. 13. **Actuators** : 4 moteurs sur AUX 1-4, DShot300 ou PWM, test moteur par moteur (position réelle $\neq$ sortie), correction du sens par la commande `dshot` ou selon la doc — jamais d'après la sérigraphie. 14. Jeu de paramètres « **attache** » chargé (§8.3) ; geofence chargée dès qu'elle existe (TP 3.7). 15. Balise : identifiant déclaré, émission vérifiée (TP 0.3).

F.6 — Phase E, finitions 16. Hélices 1045 en dernier : CW/CCW conforme à Actuators, face marquée vers le haut, bagues serrées. 17. Batterie sous sangles, centrage vérifié, pesée finale → `docs/mass-budget.md`. 18. Point d'ancrage de l'attache **sur la structure** (bras ou entretoise), paracorde, mousqueton. Puis checklist §8.4 → TP 3.5.

F.7 — Les cinq fautes qui coûtent un composant : court + / – non testé · PM02 à l'envers · TX sur TX · hélices montées avant la config · fils ESC sur MAIN au lieu d'AUX.

G. Checklist de session terrain

§8.4 (sac, pré-vol, pendant, après), imprimée et laissée dans le sac, avec le nom du jeu de paramètres à charger.

Quand tu attaques un TP, relis la section correspondante de la Partie I ; quand un TP casse le calendrier, le §10 arbitre et le bilan de sprint décide.